

SPM Module 2: Partitioned Hash Join with Duplicates

Gabriele Marsili

1 Parallelization Strategy

The partitioned hash join pipeline consists of five phases: *Histogram*, *Prefix Sum*, *Scatter*, *Join Local*, and a final sequential *Accumulation*. Each phase has distinct computational and memory characteristics that determine whether parallelism is beneficial and which strategy to adopt.

Thread Team and Barrier Synchronization

A single team of k threads is launched once and reused across all phases via `std::barrier` (C++20). An alternative design would use a thread pool with a task queue, but it would introduce unnecessary indirection: because all threads must finish each phase before the next one can begin — due to data dependencies between histogram, prefix sum, scatter, and join — a full barrier is required at every phase boundary regardless. The barrier approach directly models this bulk-synchronous structure and avoids the overhead of thread pool book-keeping (task submission, queue polling, wakeup latency). Spawning and joining threads at each phase boundary would inflate the sequential component f in Amdahl’s model by $O(k)$ spawn/join cycles per phase. The barrier completion function runs a single delegate thread between phases to perform the sequential prefix sum and to compute per-thread scatter offsets from the local histograms.

Adaptive thread count. To prevent synchronisation overhead from dominating small workloads, the actual thread count k is:

$$k = \min(k_{\max}, \lfloor \min(NR, NS) / T_{\min} \rfloor)$$

where $k_{\max}$ is the user-requested count and $T_{\min} = 8192$ records is the minimum per-thread work threshold. Below this threshold, the cost of barrier synchronisation and cache-line traffic between threads exceeds the parallelism gain: with $N < k \cdot T_{\min}$, each thread’s scan completes in less time than the $O(k \cdot P)$ histogram-merge step executed by the barrier delegate, so synchronisation overhead dominates useful work. For $NR = 10^7$ and $k_{\max} = 20$ the formula gives $k = \min(20, 1220) = 20$; for a hypothetical tiny input of $NR = 10^4$ it would give $k = \min(20, 1) = 1$, falling back to single-threaded execution. This threshold is never reached in any of the benchmarks reported below ($NR \geq 10^6$), so all experiments run with the full requested thread count.

Histogram (Phases 0 and 2)

Each thread processes a contiguous block of $\lceil N/k \rceil$ records from relation R (or S), incrementing its own

private histogram of size P . Thread-local histograms eliminate all write contention. After the barrier, the completion delegate merges the k local histograms into a global one in $O(P \cdot k)$ time, then runs the prefix sum $O(P)$ and computes per-thread scatter offsets.

Static block distribution yields good spatial locality: each thread reads a contiguous slice of the input array, maximising cache line utilisation. The dominant cost of this phase is a sequential scan of $N \times 8$ B of input — a purely memory-bandwidth-bound operation with very low arithmetic intensity ($I \approx 0.125$ FLOP/byte). As a consequence, the histogram phase scales poorly with thread count (Section 3).

Prefix Sum

The prefix sum over the P -element global histogram is performed sequentially by the barrier completion delegate. With $P \leq 1024$ this takes at most a few microseconds and represents a negligible fraction of total time ($< 0.1\%$ in all measurements). The theoretical span of a parallel prefix sum is $O(\log P)$, but the overhead of synchronising threads for such a small workload would dwarf any gain.

Scatter (Phases 1 and 3)

Scatter rearranges records into partition-contiguous output arrays. Lock-free parallelism is possible because per-thread scatter offsets are fully determined during the barrier: thread t starts writing partition p at offset

$$\text{off}[t][p] = \text{global_begin}[p] + \sum_{j=0}^{t-1} \text{local_hist}[j][p].$$

Each thread thus writes to a non-overlapping, pre-determined region of the output array with no synchronisation needed at runtime:

```
auto cursor = offsets_R[t]; // local copy on stack
for (size_t i = r_start; i < r_end; ++i) {
    uint32_t pid = hash_key(R[i].key, shift);
    out_R[cursor[pid]++] = R[i];
}
```

Scatter is memory-bound (random writes driven by hash values), but the absence of any contention allows good parallel throughput. Partition assignment uses the Fibonacci multiply-shift hash from Module 1 (`hash_key`: $h(k) = ((k_{lo} \oplus k_{hi}) \cdot A_{32}) \gg s$, $A_{32} = \lfloor 2^{32}/\varphi \rfloor$) rather than the reference $k \bmod P$, giving uniform partition sizes even for structured key sets.

Join Local (Phase 4)

Each partition can be joined independently: a flat open-addressing hash table `FlatCountMap` is built on the R

slice, then the S slice is probed. This is an embarrassingly parallel workload. At the throughput-optimal $P = 512$, each per-partition `FlatCountMap` occupies ≈ 1 MB and is processed one at a time per thread; the table fits in the per-core L3, making this phase compute-bound (hash arithmetic and cache-resident lookups) rather than DRAM-bound. Partitions are distributed cyclically: thread t owns partitions $t, t+k, t+2k, \dots$, requiring neither scheduling logic nor atomics.

```
for (uint32_t pid = t; pid < P; pid += nt) {
    size_t rb = global_begin_R[pid], re = rb + global_hist_R[pid];
    size_t sb = global_begin_S[pid], se = sb + global_hist_S[pid];
    if (rb == re || sb == se) continue;
    FlatCountMap cntR(re - rb); // build on R slice
    for (size_t i = rb; i < re; ++i) cntR.increment(out_R[i].key);
    for (size_t i = sb; i < se; ++i) // probe with S
        local.join_count += cntR.count(out_S[i].key);
    // checksum fields omitted for clarity
}
```

For the Fibonacci partitioning hash, partition sizes are statistically near-equal, so cyclic assignment achieves the same load balance as LPT scheduling — with zero scheduling overhead. LPT would require sorting all P partition sizes ($O(P \log P)$) before distribution; since sizes are near-uniform, that cost yields no benefit. Per-thread result accumulators are padded to 64 bytes (`alignas(64)`) to prevent false sharing. After the barrier, the main thread reduces the partial results.

Summary

Phase	Strategy	Bottleneck
Histogram	Thread-local + merge	Memory BW
Prefix Sum	Sequential	Negligible
Scatter	Lock-free offsets	Memory BW
Join Local	Cyclic distribution	Compute
Accumulate	Sequential	Negligible

2 Correctness Verification

Correctness is verified at two levels. For small inputs ($NR \leq 500$, $NS \leq 500$), both the sequential and parallel binaries automatically run a naive $O(N^2)$ reference join and compare `join_count`, `checksum1`, and `checksum2`:

```
./hashjoin_par -nr 50 -ns 80 -seed 13 \
-max-key 8 -p 4 -t 4
# output: naive_verify=PASS
```

For large inputs, the same three output values produced by the sequential and parallel binaries are compared across all tested thread counts. Since `checksum1` and `checksum2` use independent hash multipliers, the probability of a false positive is negligible. The verification is outside the measured region and does not affect reported timings.

3 Performance Results

All experiments were run on a node of the `spmcluster`: Intel Xeon E5-2640 v2 @ 2.00 GHz, 2 sockets $\times$ 8

physical cores (16 physical, 32 with Hyper-Threading), NUMA architecture. Default parameters: `seed=42`, $NS = 2 \times NR$, `max_key=1M`, $P=128$; best of 5 repetitions, no CPU pinning (run-to-run variation $\leq 3\%$). Strong and weak scaling fix $P = 128$ to isolate the effect of thread count from partition count; the partition sensitivity analysis (below) independently characterises the effect of P and identifies $P = 512$ as the throughput-optimal value. Thread counts $p \in \{1, 2, 4, 8, 12, 16, 20, 22, 24, 26, 28, 30, 32\}$ cover the physical-core regime ($p \leq 16$) and the Hyper-Threading regime ($p > 16$) with gaps of at most $\Delta p = 4$ between consecutive points; the phase breakdown additionally includes $p = 14$ to sample performance just below the 16-physical-core limit.

Strong Scaling

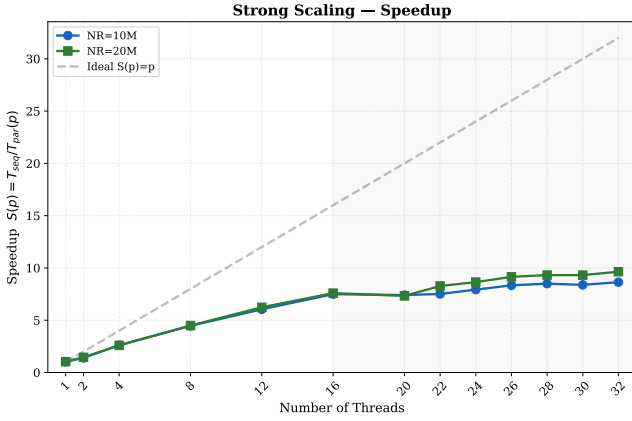
p	NR=10M		NR=20M	
	$S(p)$	$E(p)$	$S(p)$	$E(p)$
seq	1.00	100%	1.00	100%
1	1.03	103%	1.04	104%
2	1.40	70%	1.45	72%
4	2.61	65%	2.60	65%
8	4.46	56%	4.49	56%
12	6.04	50%	6.24	52%
16	7.49	47%	7.59	47%
20	7.41	37%	7.33	37%
22	7.51	34%	8.27	38%
24	7.93	33%	8.65	36%
26	8.34	32%	9.15	35%
28	8.50	30%	9.32	33%
30	8.38	28%	9.32	31%
32	8.64	27%	9.64	30%

Shaded rows: HT regime ($p > 16$). $S(1) > 1.0$: the sequential and parallel binaries are distinct executables with different startup and runtime overheads; their best-of-5 minimum timings can diverge systematically by up to $\approx 4\%$ (observed: $+2.6\%$ for NR=10M, $+4.0\%$ for NR=20M). Within-run variance across the 5 repetitions is $\leq 3\%$. The dip at $p = 20$ relative to $p = 16$ is a NUMA effect: threads crossing onto the second socket incur higher remote-memory latency (Section 4).

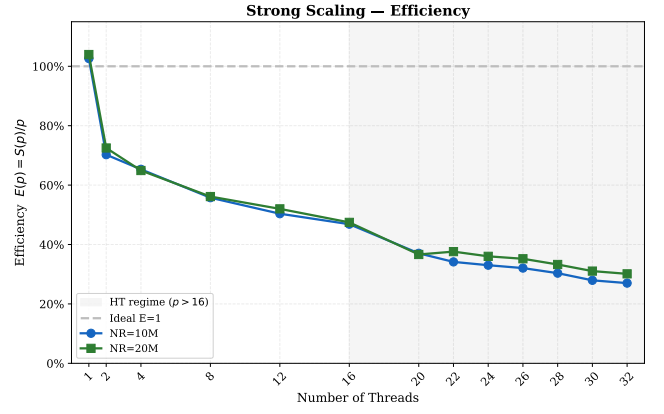
Weak Scaling

Each thread processes 1M records of R (and 2M of S). $WSE = T(1)/T(p)$; ideal is 1.0. The $p = 1$ parallel baseline (≈ 78 ms on NR=1M) incurs negligible overhead from the barrier infrastructure.

p	NR	T (ms)	WSE
1	1M	78	1.00
2	2M	106	0.74
4	4M	112	0.70
8	8M	131	0.60
10	10M	145	0.54
14	14M	155	0.51
20	20M	206	0.38
24	24M	209	0.37
32	32M	238	0.33



(a) Speedup $S(p) = T_{\text{seq}}/T_{\text{par}}(p)$.



(b) Efficiency $E(p) = S(p)/p$, declining from 70% at $p = 2$ to 27% at $p = 32$ (NR=10M). Shaded region: Hyper-Threading regime ($p > 16$).

Figure 1: Strong scaling — both problem sizes (NR=10M and NR=20M) follow nearly identical profiles.

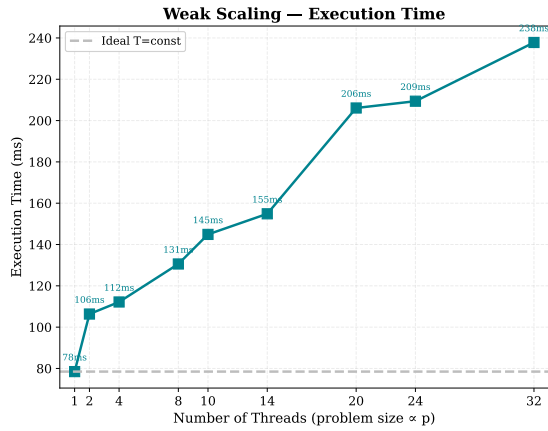
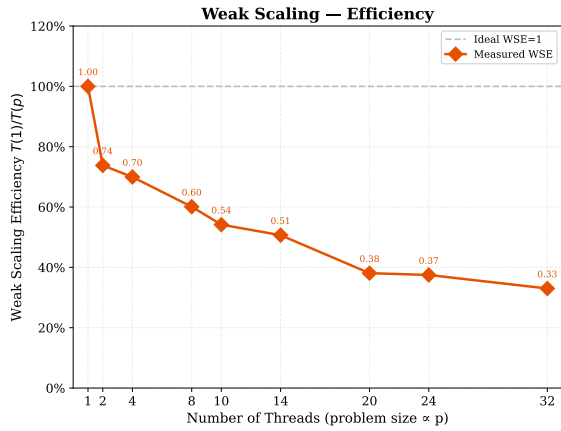


Figure 2: Weak scaling: efficiency (left) and absolute execution time (right). WSE declines monotonically; the dominant cost is scatter, whose random-write pattern saturates DRAM bandwidth as problem size grows with thread count (Section 4).

Phase Breakdown

Phase	T_{seq}	$S(14)$	$S(20)$	$S(24)$	$S(32)$
Histogram R+S	51 ms	2.0×	2.4×	3.1×	2.7×
Scatter R+S	409 ms	7.8×	9.7×	11.3×	13.7×
Join Local	309 ms	8.4×	7.2×	8.7×	8.0×
End-to-end	769 ms	6.7×	7.2×	8.7×	8.8×

Individual phase times from the pipeline’s internal timer; T_{seq} (769ms) is their sum at $p = 1$. Output buffers (out_R/out_S) are pre-allocated before timing begins; end-to-end speedup uses this sum as baseline. The sequential merge of thread-local histograms and the prefix-sum computation ($O(P \cdot k)$, a few μs) are executed by the barrier delegate between phases and are *not* included in any individual phase time; their contribution is negligible at the measured granularity. Note: the phase breakdown baseline (769ms) differs slightly from the strong scaling outer-timer baseline (766 ms), which is measured by the dedicated scaling benchmark. The $p = 1$ phase breakdown was measured twice (two consecutive diagnostic runs to verify stability: 769ms and 753ms); the first (higher) value is used as baseline, giving a conservative speedup estimate within the stated $\leq 3\%$ run-to-run variance.

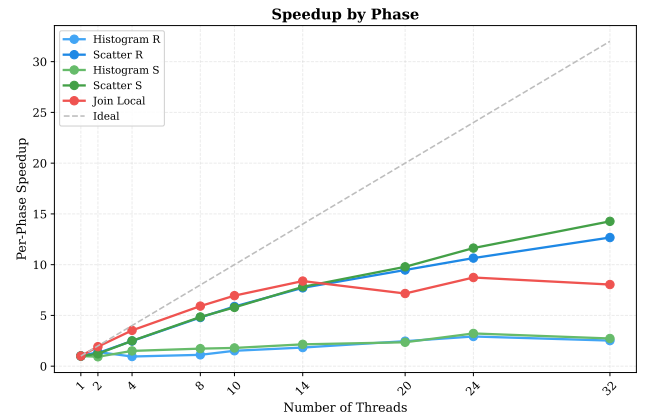


Figure 3: Per-phase speedup across $p = 1$ –32. Join Local and Scatter scale well; Histogram is memory-bound.

4 Discussion

Strong Scaling and Amdahl’s Law

Fitting Amdahl’s law $S(p) = 1/(f + (1-f)/p)$ to the full dataset yields $f \approx 0.078$, giving $S_{\infty} = 1/f \approx 12.9\times$ (Figure 4). The measured peaks are $8.64\times$ (NR=10M,

$p = 32$) and $9.64\times$ (NR=20M, $p = 32$) (Figures 1a–1b). The gap between the Amdahl prediction and the measured peak is explained by memory-bandwidth saturation: as threads increase, DRAM bandwidth is the binding constraint rather than the serial fraction.

The efficiency drops from 70% at $p = 2$ to 27% at $p = 32$ (NR=10M). With the granular thread sweep, the NUMA crossover at $p = 16 \rightarrow 20$ is directly observable: speedup reaches $7.49\times$ ($7.59\times$ for NR=20M) at $p = 16$ (last physical-core point), then *dips* to $7.41\times$ ($7.33\times$ for NR=20M) at $p = 20$ before recovering as HT threads improve scatter throughput. This transient regression is consistent with threads on the second socket accessing remote memory at higher latency, a hypothesis supported by the larger dip for NR=20M (larger working set, more NUMA traffic). From $p = 16$ to $p = 32$, the end-to-end time improves by only $\approx 13\%$ ($102\text{ms} \rightarrow 89\text{ms}$ for NR=10M; $\approx 21\%$ for NR=20M: $207\text{ms} \rightarrow 163\text{ms}$) despite doubling the logical thread count, making $p = 12\text{--}16$ (physical cores only) the practical sweet spot.

Effect of partition count ($P = 512$). At the throughput-optimal $P = 512$ (Partition Count Sensitivity subsection, below), the implementation achieves $11.4\times$ (NR=10M) and $11.5\times$ (NR=20M) speedup at $p = 32$, with a fitted serial fraction $f \approx 0.058$ ($S_\infty \approx 17\times$). The reduction in f relative to $P = 128$ reflects smaller per-partition `FlatCountMap` tables that fit in L3, reducing the DRAM-bound component of the join phase. At $P = 512$ the NUMA dip is entirely absent for NR=10M: speedup rises from $7.76\times$ at $p = 16$ to $9.16\times$ at $p = 20$, because finer partitioning keeps each `FlatCountMap` within the aggregate L3. For NR=20M the dip remains visible ($9.59\times \rightarrow 9.22\times$ from $p = 16$ to $p = 20$), confirming that the larger working set still generates significant NUMA traffic even at $P = 512$. Notably, the dip magnitude ($\Delta S = 0.37$) exceeds that at $P = 128$ ($\Delta S = 0.26$), indicating that finer partitioning does not reduce NUMA penalties for NR=20M.

Hyper-Threading regime ($p = 20\text{--}32$). The phase breakdown reveals a counter-intuitive split at $p > 16$: scatter continues to improve (R+S combined: $9.7\times \rightarrow 13.7\times$ from $p = 20$ to $p = 32$) while join has a local maximum at $p = 14$ ($8.4\times$), dips to $7.2\times$ at $p = 20$ (HT onset), recovers to $8.7\times$ at $p = 24$, then settles at $8.0\times$ at $p = 32$. Scatter is memory-bandwidth-bound — HT threads increase memory-level parallelism, so additional logical cores help. Join Local (hash-table lookups with `FlatCountMap`) is compute-bound: HT pairs share execution ports on the same physical core, so competing hash-lookup streams contend for ALU and L1/L2 resources rather than adding capacity. Overall, end-to-end speedup improves only moderately ($7.2\times \rightarrow 8.8\times$) because the join plateau offsets much of the scatter gain. The Join Local phase breakdown is measured from a single diagnostic run (not aggregated over repetitions), so the per-point fluctuation ($7.2\times \rightarrow 8.7\times \rightarrow 8.0\times$ across $p = 20, 24, 32$) lies within typical single-run noise and should be read as a plateau trend rather than a monotone sequence. The qualitative split between a memory-bound scat-

ter (steadily improving through the HT regime) and a compute-bound join (plateauing) is robust across repetitions of the end-to-end benchmark.

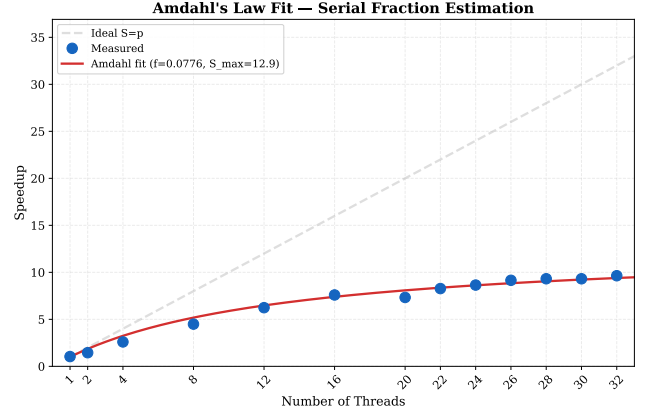


Figure 4: Amdahl’s law fit (fit computed on both NR=10M and NR=20M; NR=20M curve shown, $P=128$). Estimated serial fraction $f \approx 0.078$; theoretical $S_\infty \approx 12.9\times$. The measured peak ($9.64\times$ at $p = 32$) falls below the prediction due to memory-bandwidth saturation.

Weak Scaling and WSE Interpretation

The $p = 1$ parallel binary takes $\approx 78\text{ms}$ on NR=1M with negligible overhead from the barrier infrastructure. $\text{WSE} = T(1)/T(p)$ decreases monotonically: 0.74 at $p = 2$, 0.54 at $p = 10$, 0.33 at $p = 32$ (Figure 2).

The primary cause is scatter: as the problem size grows proportionally with thread count, the random-write pattern of scatter generates an increasing volume of DRAM traffic that cannot be absorbed by more cores (bandwidth is shared, not replicated). Histogram and join scale somewhat better because their access patterns benefit from per-thread private structures (thread-local histograms) or fit within the aggregate L3 capacity at low thread counts (for $p \leq 2$ each per-partition `FlatCountMap` is $\leq 512\text{KB}$; at $p = 4$ it reaches $\approx 1\text{MB}$ — in both cases well within the 20MB aggregate L3 (at most $p \times 1\text{MB} \ll 20\text{MB}$)). WSE drops most sharply between $p = 14$ and $p = 20$ ($0.51 \rightarrow 0.38$, $\Delta=0.13$), then levels off ($p = 20 \rightarrow 32$: $\Delta=0.05$). The step coincides with the transition into the HT regime and growing NUMA penalties: with two sockets, threads on the second socket typically access remote memory at appreciably higher latency.

Effect of partition count on weak scaling. At the throughput-optimal $P = 512$, weak scaling improves appreciably: $T(1) \approx 90\text{ms}$ (NR=1M), $\text{WSE} = 0.45$ at $p = 32$ versus 0.33 at $P = 128$ — a $\approx 35\%$ improvement in scaling efficiency; in absolute execution time at $p = 32$ the gain is $\approx 15\%$ ($238 \rightarrow 202\text{ms}$). The finer partitioning also produces a transient cache-benefit at $p = 4$ ($\text{WSE} = 0.87 > \text{WSE}(p=2) = 0.79$): more `FlatCountMap` tables fit within the aggregate L3, briefly improving efficiency before DRAM-bandwidth pressure dominates.

Phase-Level Analysis

The three computational phases (Figure 3) scale very differently, and understanding why is central to the gap between the Amdahl prediction and the measured end-to-end speedup.

Histogram is the weakest scaler ($2.4\text{--}3.1\times$ in the HT regime). Each thread performs one counter increment per 8-byte record, so the memory bus is the bottleneck regardless of core count. At $\approx 6.6\%$ of sequential time (51 ms of 769 ms), histogram contributes a small but measurable fraction of the overall $f \approx 0.078$.

Scatter dominates the sequential baseline ($\approx 53\%$, 409 ms of 769 ms) and, unlike histogram, keeps improving through the HT regime ($9.7 \rightarrow 13.7\times$ from $p = 20$ to $p = 32$). The lock-free design with pre-computed offsets eliminates contention entirely; the only limit is DRAM write bandwidth, and HT threads help here because they increase memory-level parallelism rather than competing for ALU resources.

The Join Local phase exhibits a contrasting scaling pattern. At $P = 128$ with $\text{NR}=10\text{M}$, each per-partition **FlatCountMap** occupies $\approx 4\text{ MB}$ ($256\text{K slots} \times 16\text{ B}$); with 20 threads active, the combined working set reaches $20 \times 4\text{ MB} = 80\text{ MB}$, far above the 20 MB shared L3. The phase scales near-linearly up to $p = 14$ ($8.4\times$) but then plateaus: HT pairs share execution ports on the same physical core, so adding logical threads does not add compute capacity. At the optimal $P = 512$, each table shrinks to $\approx 1\text{ MB}$ (64K slots) and the aggregate $\approx 20\text{ MB}$ fits roughly within the shared L3, which explains the performance peak observed in the partition sensitivity analysis (Partition Count Sensitivity subsection, below).

Partition Count Sensitivity

With $P \in [16, 4096]$ (at $p = 20$, $\text{NR}=10\text{M}$), execution time reaches a minimum at $P = 512\text{--}1024$ ($\approx 86\text{ ms}$), with a variation of $< 1.6\times$ across the full range. At $P = 16$ each partition holds $\approx 625\text{K}$ records and the **FlatCountMap** occupies $\approx 32\text{ MB}$, spilling into DRAM. The optimum at $P = 512$ corresponds to $\approx 20\text{K}$ records/partition, with the hash table ($\approx 1\text{ MB}$) fitting well within the shared 20 MB L3. Beyond $P = 1024$ the overhead of repeated **FlatCountMap** allocation/deallocation cycles grows: at $P = 4096$ each thread performs $\lceil P/p \rceil = \lceil 4096/20 \rceil = 205$ constructor/destructor calls per join phase (vs. $\lceil 512/20 \rceil = 26$ at $P = 512$), increasing TLB pressure and loop overhead, raising time to 89 ms at $P = 2048$ and 94 ms at $P = 4096$. A non-monotonic bump at $P = 32$ (130 ms, slower than $P = 16$) arises from a two-effect transition. At $P = 16 < p=20$: the cyclic assignment exhausts all partitions after 16 threads, leaving 4 threads idle throughout the join phase; the 16 active threads each execute one build-probe cycle on a $\approx 32\text{ MB}$ **FlatCountMap** (single allocation, DRAM-bound). At $P = 32 (\gtrsim p=20)$: all 20 threads are active but 12 of them process two partitions sequentially, each requiring a fresh 16 MB **FlatCountMap** allocation; the maximum thread work-

load ($\approx 625\text{K}$ records) is nearly identical to $P=16$, yet the extra malloc-free cycle between partitions adds construction overhead without any cache benefit, since 16 MB tables are still firmly DRAM-bound. From $P = 64$ onwards, per-partition tables shrink below $\approx 8\text{ MB}$ and begin to overlap with the shared 20 MB L3, so additional allocation cycles are more than compensated by reduced DRAM traffic, restoring monotone improvement. The minimum lies in $P \in [512, 1024]$ ($\approx 86\text{ ms}$); timing improves by $\approx 20\%$ from $P = 128$ (107 ms) to $P = 512$, so the choice of P is meaningful in the sub-optimal range (Figure 5).

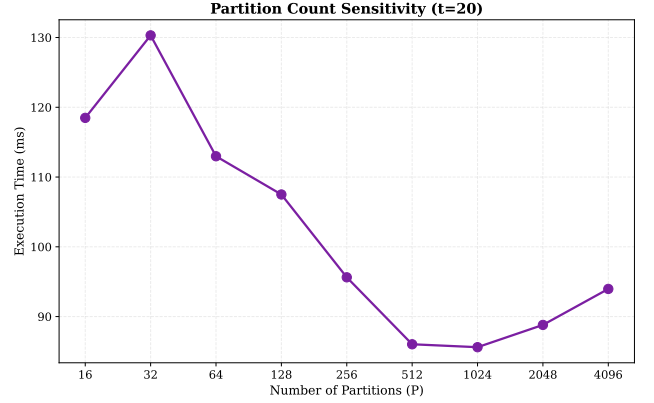


Figure 5: Partition count sensitivity ($p = 20$, $\text{NR}=10\text{M}$). Minimum at $P = 512\text{--}1024$; non-monotonic bump at $P = 32$ reflects a cache-occupancy transition.

Duplicate Density

Sequential time is essentially constant for $\text{max_key} \leq 100\text{K}$ ($\approx 697\text{--}721\text{ ms}$): with many duplicates, few distinct keys keep the per-partition **FlatCountMap** sparsely occupied and most probes terminate quickly. Speedup peaks at $\text{max_key}=10\text{K}$ ($7.5\times$), where key values distribute uniformly across $P = 128$ partitions and per-partition tables are neither too sparse nor too densely populated. At $\text{max_key}=10\text{M}$ (near-unique keys) the hash table is fully populated, increasing sequential time by $+29\%$ ($\approx 708\text{ ms} \rightarrow 915\text{ ms}$, relative to the peak-speedup point at $\text{max_key}=10\text{K}$) while parallel execution time rises by $+42\%$ ($\approx 94\text{ ms} \rightarrow 133\text{ ms}$)—a larger relative increase—because at $P = 128$ each per-partition **FlatCountMap** occupies $\approx 4\text{ MB}$, well above the per-core L3 share, so the working set remains DRAM-bound at any key density and the parallel version cannot exploit sparser table occupancy. As a result, speedup at $\text{max_key}=10\text{M}$ settles at $\approx 6.9\times$, modestly below the $7.5\times$ peak, showing that the partitioned approach works well regardless of key distribution. At the optimal $P = 512$, per-partition tables shrink to $\approx 1\text{ MB}$ and can partially reside in L3 at near-unique densities, explaining the substantially higher speedup observed in that regime (Figure 6).

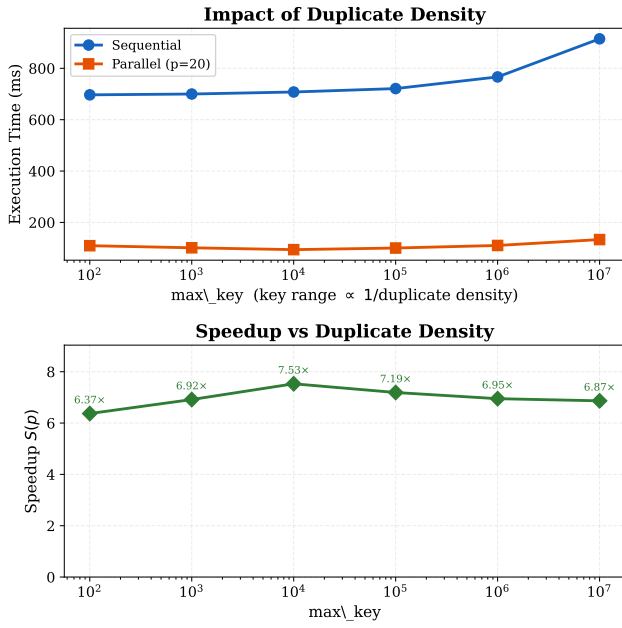


Figure 6: Impact of duplicate density ($p = 20$, $NR=10M$, $P = 128$). Top: absolute execution time (seq and par). Bottom: speedup vs. **max_key**. Speedup peaks at **max_key**=10K (7.5 $\times$); drops to 6.4 $\times$ at **max_key**=100 due to partition load imbalance (100 distinct keys for 128 partitions); settles at 6.9 $\times$ at **max_key**=10M (near-unique) because per-partition tables remain DRAM-bound at $P = 128$ regardless of key density.